

Rust/C++ Course Bird's-Eye Map

Teaching detours, and a topic map for a non-linear, C/C++-rooted Rust course.

**Rust not as foreign syntax.
Its a systems reasoning with a compiler that demands
proof.**

C/C++ intuition

Rust proof obligations

Fix-It exercises

GPU boundary

What does the compiler know?

A course for people who already think in memory

The goal is not to replace C++ experience. It is to interrogate it.

The audience

- experienced C/C++ developers
- already understand cost, layout and ABI pressure
- may know the rules but not always name them

Start from respect.

The Rust move

- make implicit ownership reasoning explicit
- turn convention into types where possible
- move some proof from review into compilation

Rust is not magic.

The real prize

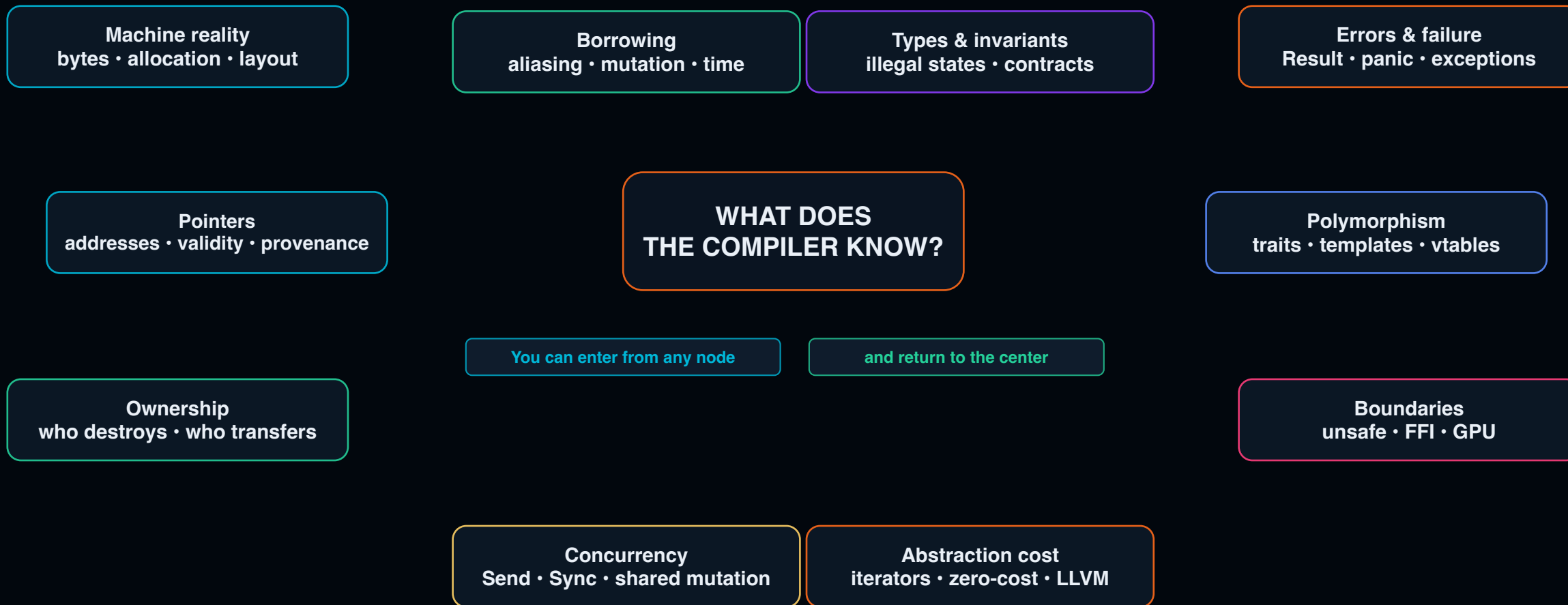
- better Rust programmers
- better C++ programmers
- stronger instincts about lifetime, aliasing and invariants

The why precedes the pattern.

When Rust complains, ask: “What fact is the compiler unable to prove?”

The whole course as one mental model

Every detour is a route back to one question: what fact must be true for this program to be valid?



Think in reusable stacks, not a fixed syllabus

Each stack can be entered from a class question, a compiler error, a Fix-It, or a C++ comparison.

Core stacks already built

- Memory literacy
- Pointers
- Ownership & moves
- Aliasing, borrowing & lifetimes
- Mutation & concurrency
- Types & invariants
- Unsafe + macros/attributes

Use as needed.

Missing stacks to add

- Errors & failure
- Traits/generics/polymorphism
- Representation/layout/zero-cost
- Iterators & ranges
- FFI boundaries
- Async/Futures/Pin
- Toolchain/testing/profiling

Build these next.

Course glue

- Bird's-eye topic map
- Fix-It routing map
- C++ comparison bridges
- Day 3 GPU bridge
- Final synthesis: what does the compiler know?

Use between stacks.

A flight system for a non-linear class

You do not need to follow the map. You need a map good enough to leave it.

Trigger

- student asks “why?”
- compiler error appears
- C++ analogy becomes dangerous
- performance question arises

Class chooses entry point

Detour

- pull the relevant stack
- show a failure case
- name the hidden invariant
- compare to C++

Short, sharp module

Fix-It

- predict failure mode
- explain semantic gap
- make smallest Rust fix
- compare solution

Make it concrete

Return

- back to syntax slide
- back to roadmap
- back to GPU example
- back to compiler question

Close the loop

Anchor

- What does the compiler know?
- What must the programmer prove?

Always

The map is not the route. The map lets you improvise without losing the course.

Errors, exceptions, Result and failure semantics

The deep issue: which failures are part of the function contract?

C++ questions:

- return code, exception, `std::optional`, `std::expected`?
- what happens during unwinding?
- which failures are recoverable?
- which failures violate invariants?

Rust questions:

- `Option<T>` for absence
- `Result<T,E>` for recoverable failure
- `panic!` for violated assumptions
- `?` as propagation, not invisibility

Slides to include

- error as API surface
- recoverable vs unrecoverable
- C++ exceptions vs Rust Result
- RAI and Drop during failure
- panic: unwind or abort

Do not teach `?` as syntax. Teach it as explicit propagation of a typed failure channel.

Traits, generics and polymorphism

The C++ bridge: templates, concepts, virtual dispatch and type erasure.

C++ mental model

- templates: compile-time substitution
- concepts: constraints named clearly
- virtual: runtime dispatch
- type erasure: hide concrete type

Cost model matters.

Rust mental model

- trait bounds: compile-time contracts
- impl Trait: concrete but hidden type
- dyn Trait: vtable object
- associated types: part of the contract

Same questions, different spelling.

Key slides

- monomorphization
- object safety
- vtable boundary
- blanket impls
- orphan rule

Why the rules exist

Fix-Its

- trait bound missing
- wrong return abstraction
- borrowed trait object
- associated type confusion

Make cost visible

Representation, layout and zero-cost abstraction

For C/C++ people, the bytes are never an implementation detail for long.

A type is not only operations. It is representation, alignment, validity and meaning.

Memory layout

- stack vs heap
- alignment and padding
- String / Vec descriptors
- slices and fat pointers

Representation control

- repr(Rust) vs repr(C)
- ABI promises
- niche optimization
- Option<&T> intuition

Zero-cost claim

- monomorphization
- inlining
- LLVM visibility
- where abstraction disappears

C++ bridge

- struct layout
- std::string shape
- templates
- optimizer trust

Iterators, ranges and abstraction cost

The deep issue is not syntax. It is whether a pleasant abstraction still compiles to the loop you meant.

C++20 style:

```
auto r = xs | std::views::filter(pred)
           | std::views::transform(f);

for (auto x : r) { use(x); }
```

Rust style:

```
xs.iter()
  .filter(|x| pred(x))
  .map(|x| f(x))
  .for_each(|x| use(x));
```

Teach the why

- lazy pipeline
- ownership of iteration
- borrowing during traversal

Inspect the cost

- MIR/LLVM/assembly glimpses
- inlining and specialization
- bounds checks and elision

Fix-It angle

- iterator consumed
- lazy iterator unused
- borrowed vs owned item

FFI and boundaries: where guarantees end

Rust's safety story is strongest when the boundary is explicit. C/C++ interop is the proof boundary.

Boundary facts

- layout agreement
- ownership transfer rules
- who frees memory
- nullable vs non-null
- unwind behavior

Rust tools

- `extern "C"`
- `repr(C)`
- raw pointers
- `CString` / `CStr`
- unsafe wrapper API

C++ tools

- headers
- ABI discipline
- RAII wrapper
- `pimpl`
- opaque handle

Good lesson

- unsafe inside
- safe API outside
- document ownership
- test the boundary

FFI is not “turn safety off.” It is “move the proof obligation to a small, inspectable border.”

Async, futures and Pin

Only teach this after ownership, moves, borrowing and address stability are alive in the room.

async is easier when students already understand that moving a value can change where it lives.

What async really

creates

- state machine
- captured values
- suspension points
- borrow across await?

Why Pin appears

- address-sensitive state
- self-references
- future may be polled later
- movement restrictions

C++ comparison

- coroutines
- lifetime of frame
- captures
- executor boundary

Keep modest

- conceptual only
- one small demo
- avoid runtime rabbit hole

Toolchain, crates, modules, testing and profiling

Practical Rust is not only the language. It is also the build/test/diagnostic culture.

Cargo & crates

- package model, features, dependency boundaries

Modules & visibility

- public API surface, privacy as design tool

Testing

- unit/integration/doc tests, property-style thinking

Diagnostics

- clippy, rustfmt, compiler messages as teaching material

Profiling

- criterion/perf/flamegraph mindset, not folklore

Publishing

- semver, docs, crate hygiene

This stack keeps the course from becoming language theory without engineering practice.

GPU: a proving ground for the whole model

Day 3 becomes stronger when GPU programming is framed as ownership across address spaces.

Resource reality

- host memory
- device memory
- unified memory
- streams/events
- kernel launch boundary

Rust questions

- who owns the buffer?
- who may mutate?
- what lifetime spans the kernel?
- where is unsafe justified?

C++ questions

- RAI for CUDA handles
- copy/move of buffers
- synchronization discipline
- error checking

Course payoff

- unsafe boundary is meaningful
- ownership is visible
- lifetime bugs are concrete
- performance is not abstract

Use the 100 exercises as the heartbeat of the course

Each exercise should answer: what did Rust reject, and what would C++ require the programmer to remember?

01 Ownership & Moves

02 Borrowing & References

03 Strings & Slices

04 Structs, Enums & Patterns

05 Option, Result & Errors

06 Collections & Iterators

07 Traits & Generics

08 Lifetimes

09 Smart Pointers & Interior Mutability

10 Threads, Channels & Synchronization

Use one Fix-It whenever the room needs proof that the concept is not just a slide.

A practical trigger table for live teaching

Use the student's confusion as your navigation signal.

"Why can't I use it after this line?"	Ownership & moves
"But it is just a reference."	Pointers → borrowing → lifetimes
"Why one mutable reference?"	Aliasing + mutation
"Why Option instead of null?"	Types & invariants
"Why Result everywhere?"	Error/failure semantics
"Why not just inheritance?"	Traits/generics/polymorphism
"Is this abstraction slow?"	Representation/layout/zero-cost
"Why unsafe here?"	Unsafe/FFI boundary
"Why does async need Pin?"	Moves + address stability + async

A roadmap that supports detours

The order is not a cage. It is a set of default gravity wells.

Day 1 — Model

- opening thesis + memory reality
- minimum viable syntax
- ownership + moves
- pointers + borrowing
- lifetimes as time claims
- types/invariants + errors
- Fix-Its throughout

Build the mental model

Day 2 — Consequences

- traits/generics
- representation/layout
- iterators/zero-cost
- interior mutability
- concurrency
- unsafe + FFI
- macros/attributes + tooling

Explore what follows

Day 3 — Application

- Rust GPU ecosystem
- host/device ownership
- kernel safety boundary
- interop with CUDA C++
- benchmarking
- capstone kernel
- return to compiler map

Apply the model under pressure

Make the deck navigable under pressure

The best slide library lets you jump without hunting.

Name stacks by question

- what owns it?
- who can mutate?
- how long is valid?
- what states exist?
- where does proof stop?

Give each stack a map slide

- entry points
- C++ bridge
- Rust concept
- Fix-It links
- exit slide

Use anchors

- compiler knows?
- what moved?
- who aliases?
- what invariant?

Keep detours short

- 5–12 slides
- one thesis
- one example
- one Fix-It

What does the compiler know?

This should be the closing deck and the recurring anchor.

Fact	C++	Rust
Object exists	mostly yes	yes
Ownership transferred	partly by convention/types	language rule
Borrow still valid	often programmer proof	borrow checker where expressible
Aliasing permits mutation	programmer/library discipline	& / &mut discipline
Invariant holds	constructors/API discipline	types + API + unsafe contracts
Thread sharing legal	discipline + library	Send/Sync + synchronization
FFI contract valid	programmer proof	programmer proof
Business logic correct	programmer proof	programmer proof

The goal is not to stop thinking. The goal is to decide which facts the compiler should think about with us.

If you create more decks, create them in this order

The order follows classroom leverage: highest conceptual payoff first.

1	Errors & failure semantics	Connects Option/Result to C++ exceptions/std::expected and API contracts.
2	Traits/generics/polymorphism	Necessary for idiomatic Rust and strong C++20 concepts comparison.
3	Representation/layout/zero-cost	Satisfies systems programmers: what are the bytes and costs?
4	Iterators/ranges	Connects ownership, borrowing and performance in everyday code.
5	FFI boundaries	Prepares for unsafe, C++ interop and GPU code.
6	Async/Futures/Pin	Useful but only after address stability and lifetimes are established.
7	Toolchain/testing/profiling	Engineering practice stack; essential, but less philosophical.

The course is about systems thinking

Rust is the new language; C/C++ is the home territory; the machine is the common ground.

**Most languages cook with water.
The interesting question is what they force you to name.**

C/C++ gives power

- direct model of memory
- control over representation
- responsibility stays with the programmer

Rust gives pressure

- name ownership
- prove borrowing
- quarantine unsafe assumptions

The class gives judgment

- when to trust types
- when to inspect bytes
- when the programmer must still prove

Use the map. Leave the map. Return to the map.